

E-COMMERCE CONVERSION RATE ANALYSIS

Complete Data Analytics Project – Python

Project Objective: Analyze an e-commerce website's visitor/session data to calculate conversion rate, understand customer behavior, compare conversion across devices, traffic sources and user types, and identify factors associated with successful purchases. The project is designed as a complete, runnable Python analytics project.

1. Project Workflow

Data Collection → Data Cleaning → Exploratory Data Analysis (EDA) → Conversion Rate Calculation → Segment Analysis → Visualization → Statistical/Correlation Analysis → Business Insights → Final Report

2. Dataset Description

The code below creates a realistic sample e-commerce dataset automatically, so the project can be executed even without downloading an external dataset. Each row represents one website session. Important fields include visitor type, device, traffic source, pages viewed, session duration, bounce rate, cart activity, and whether the session resulted in a purchase.

Column	Meaning
session_id	Unique session identifier
visitor_type	New Visitor or Returning Visitor
device	Desktop, Mobile, or Tablet
traffic_source	Organic, Paid Ads, Social Media, Direct, Referral
pages_viewed	Number of pages viewed
session_duration	Session duration in minutes
bounce_rate	Percentage of visitors leaving quickly
cart_added	1 if item was added to cart, otherwise 0
purchase	1 if purchase occurred, otherwise 0

3. Installation / Requirements

```
pip install pandas numpy matplotlib seaborn
```

4. Complete Python Code

```
# =====
# E-COMMERCE CONVERSION RATE - DATA ANALYTICS PROJECT
# =====
# Technologies: Python, Pandas, NumPy, Matplotlib, Seaborn
# =====

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# -----
# 1. PROJECT SETTINGS
# -----
np.random.seed(42)

OUTPUT_DIR = "ecommerce_outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# -----
# 2. CREATE SAMPLE DATASET
# -----
n = 5000

visitor_type = np.random.choice(
    ["New Visitor", "Returning Visitor"],
    size=n,
    p=[0.65, 0.35]
)

device = np.random.choice(
    ["Desktop", "Mobile", "Tablet"],
    size=n,
    p=[0.45, 0.45, 0.10]
)

traffic_source = np.random.choice(
    ["Organic", "Paid Ads", "Social Media", "Direct", "Referral"],
    size=n,
    p=[0.30, 0.20, 0.20, 0.20, 0.10]
)

pages_viewed = np.random.poisson(lam=5, size=n) + 1
pages_viewed = np.clip(pages_viewed, 1, 25)

session_duration = np.random.gamma(
    shape=2.2, scale=2.5, size=n
)
session_duration = np.round(
    np.clip(session_duration, 0.2, 30), 2
)

bounce_rate = 100 - (
    pages_viewed * 3.0 +
    session_duration * 1.7 +
    np.random.normal(0, 10, n)
)
bounce_rate = np.round(
    np.clip(bounce_rate, 5, 95), 2
)

# Cart probability depends on engagement
cart_probability = (
    0.04
    + 0.018 * pages_viewed
    + 0.012 * session_duration
    - 0.0012 * bounce_rate
)
cart_probability = np.clip(cart_probability, 0.01, 0.75)

cart_added = np.random.binomial(1, cart_probability)

# Purchase probability depends on cart and engagement
purchase_probability = (
    0.005
    + 0.10 * cart_added
    + 0.004 * pages_viewed
)
```

```

        + 0.002 * session_duration
        - 0.0008 * bounce_rate
    )

# Returning visitors convert slightly more often
purchase_probability += np.where(
    visitor_type == "Returning Visitor", 0.025, 0
)

# Traffic-source effects
purchase_probability += np.select(
    [
        traffic_source == "Organic",
        traffic_source == "Paid Ads",
        traffic_source == "Social Media",
        traffic_source == "Direct",
        traffic_source == "Referral"
    ],
    [0.015, 0.010, -0.005, 0.020, 0.005],
    default=0
)

# Device effects
purchase_probability += np.select(
    [
        device == "Desktop",
        device == "Mobile",
        device == "Tablet"
    ],
    [0.015, 0.000, 0.005],
    default=0
)

purchase_probability = np.clip(
    purchase_probability, 0.001, 0.85
)

purchase = np.random.binomial(
    1, purchase_probability
)

df = pd.DataFrame({
    "session_id": np.arange(1, n + 1),
    "visitor_type": visitor_type,
    "device": device,
    "traffic_source": traffic_source,
    "pages_viewed": pages_viewed,
    "session_duration": session_duration,
    "bounce_rate": bounce_rate,
    "cart_added": cart_added,
    "purchase": purchase
})

# Save raw dataset
df.to_csv(
    os.path.join(OUTPUT_DIR, "ecommerce_sessions.csv"),
    index=False
)

# -----
# 3. DATA UNDERSTANDING
# -----
print("\n===== DATASET SHAPE =====")
print(df.shape)

print("\n===== FIRST 10 ROWS =====")
print(df.head(10))

print("\n===== DATA TYPES =====")
print(df.dtypes)

print("\n===== MISSING VALUES =====")
print(df.isnull().sum())

print("\n===== DUPLICATES =====")
print(df.duplicated().sum())

print("\n===== STATISTICS =====")
print(df.describe(include="all"))

# -----

```

```

# 4. DATA CLEANING
# -----
df = df.drop_duplicates()

numeric_columns = [
    "pages_viewed",
    "session_duration",
    "bounce_rate",
    "cart_added",
    "purchase"
]

for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df["pages_viewed"] = df["pages_viewed"].fillna(
    df["pages_viewed"].median()
)

df["session_duration"] = df["session_duration"].fillna(
    df["session_duration"].median()
)

df["bounce_rate"] = df["bounce_rate"].fillna(
    df["bounce_rate"].median()
)

df["cart_added"] = df["cart_added"].fillna(0).astype(int)
df["purchase"] = df["purchase"].fillna(0).astype(int)

# -----
# 5. OVERALL CONVERSION RATE
# -----
total_sessions = len(df)
total_purchases = df["purchase"].sum()

conversion_rate = (
    total_purchases / total_sessions
) * 100

print("\n===== OVERALL CONVERSION =====")
print("Total sessions:", total_sessions)
print("Total purchases:", total_purchases)
print(f"Conversion Rate: {conversion_rate:.2f}%")

# -----
# 6. CART CONVERSION / CART RATE
# -----
cart_users = df["cart_added"].sum()

cart_rate = (
    cart_users / total_sessions
) * 100

if cart_users > 0:
    cart_to_purchase_rate = (
        df.loc[df["cart_added"] == 1, "purchase"].mean()
    ) * 100
else:
    cart_to_purchase_rate = 0

print("\n===== CART ANALYSIS =====")
print(f"Cart Addition Rate: {cart_rate:.2f}%")
print(
    f"Cart-to-Purchase Rate: "
    f"{cart_to_purchase_rate:.2f}%"
)

# -----
# 7. CONVERSION BY DEVICE
# -----
device_conversion = (
    df.groupby("device")["purchase"]
    .mean()
    .mul(100)
    .sort_values(ascending=False)
)

print("\n===== CONVERSION BY DEVICE =====")
print(device_conversion.round(2))

```

```

# -----
# 8. CONVERSION BY VISITOR TYPE
# -----
visitor_conversion = (
    df.groupby("visitor_type")["purchase"]
      .mean()
      .mul(100)
      .sort_values(ascending=False)
)

print("\n===== CONVERSION BY VISITOR TYPE =====")
print(visitor_conversion.round(2))

# -----
# 9. CONVERSION BY TRAFFIC SOURCE
# -----
source_conversion = (
    df.groupby("traffic_source")["purchase"]
      .mean()
      .mul(100)
      .sort_values(ascending=False)
)

print("\n===== CONVERSION BY TRAFFIC SOURCE =====")
print(source_conversion.round(2))

# -----
# 10. CONVERSION BY CART STATUS
# -----
cart_conversion = (
    df.groupby("cart_added")["purchase"]
      .mean()
      .mul(100)
)

print("\n===== CONVERSION BY CART STATUS =====")
print(cart_conversion.round(2))

# -----
# 11. ENGAGEMENT COMPARISON
# -----
engagement = (
    df.groupby("purchase")
      [["pages_viewed", "session_duration", "bounce_rate"]]
      .mean()
)

print("\n===== ENGAGEMENT BY PURCHASE =====")
print(engagement.round(2))

# -----
# 12. CONVERSION BY PAGE-VIEW GROUP
# -----
df["page_group"] = pd.cut(
    df["pages_viewed"],
    bins=[0, 3, 6, 10, 25],
    labels=["1-3", "4-6", "7-10", "11+"]
)

page_conversion = (
    df.groupby("page_group", observed=True)["purchase"]
      .mean()
      .mul(100)
)

print("\n===== CONVERSION BY PAGE GROUP =====")
print(page_conversion.round(2))

# -----
# 13. CONVERSION BY SESSION DURATION
# -----
df["duration_group"] = pd.cut(
    df["session_duration"],
    bins=[0, 2, 5, 10, 30],
    labels=["0-2 min", "2-5 min", "5-10 min", "10+ min"]
)

duration_conversion = (
    df.groupby("duration_group", observed=True)["purchase"]
      .mean()
      .mul(100)
)

```

```

)

print("\n===== CONVERSION BY DURATION =====")
print(duration_conversion.round(2))

# -----
# 14. CROSS ANALYSIS
# -----
device_source = pd.pivot_table(
    df,
    values="purchase",
    index="traffic_source",
    columns="device",
    aggfunc="mean"
) * 100

print("\n===== DEVICE x TRAFFIC SOURCE =====")
print(device_source.round(2))

# -----
# 15. CORRELATION ANALYSIS
# -----
corr_cols = [
    "pages_viewed",
    "session_duration",
    "bounce_rate",
    "cart_added",
    "purchase"
]

correlation = df[corr_cols].corr()

print("\n===== CORRELATION MATRIX =====")
print(correlation.round(3))

# -----
# 16. VISUALIZATION SETTINGS
# -----
sns.set_theme(style="whitegrid")

# Chart 1: Overall conversion
plt.figure(figsize=(7, 5))
plt.bar(
    ["Non-Purchase", "Purchase"],
    [
        total_sessions - total_purchases,
        total_purchases
    ]
)
plt.title("Purchase vs Non-Purchase Sessions")
plt.ylabel("Number of Sessions")
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "01_purchase_distribution.png"),
    dpi=300
)
plt.show()

# Chart 2: Conversion by device
plt.figure(figsize=(8, 5))
device_conversion.plot(kind="bar")
plt.title("Conversion Rate by Device")
plt.ylabel("Conversion Rate (%)")
plt.xlabel("Device")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "02_conversion_by_device.png"),
    dpi=300
)
plt.show()

# Chart 3: Conversion by visitor type
plt.figure(figsize=(8, 5))
visitor_conversion.plot(kind="bar")
plt.title("Conversion Rate by Visitor Type")
plt.ylabel("Conversion Rate (%)")
plt.xlabel("Visitor Type")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(

```

```

        os.path.join(OUTPUT_DIR, "03_conversion_by_visitor.png"),
        dpi=300
    )
    plt.show()

# Chart 4: Conversion by traffic source
plt.figure(figsize=(9, 5))
source_conversion.plot(kind="bar")
plt.title("Conversion Rate by Traffic Source")
plt.ylabel("Conversion Rate (%)")
plt.xlabel("Traffic Source")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "04_conversion_by_source.png"),
    dpi=300
)
plt.show()

# Chart 5: Pages viewed vs purchase
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="purchase",
    y="pages_viewed"
)
plt.title("Pages Viewed by Purchase Status")
plt.xlabel("Purchase (0 = No, 1 = Yes)")
plt.ylabel("Pages Viewed")
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "05_pages_vs_purchase.png"),
    dpi=300
)
plt.show()

# Chart 6: Session duration vs purchase
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="purchase",
    y="session_duration"
)
plt.title("Session Duration by Purchase Status")
plt.xlabel("Purchase (0 = No, 1 = Yes)")
plt.ylabel("Session Duration (minutes)")
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "06_duration_vs_purchase.png"),
    dpi=300
)
plt.show()

# Chart 7: Bounce rate vs purchase
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="purchase",
    y="bounce_rate"
)
plt.title("Bounce Rate by Purchase Status")
plt.xlabel("Purchase (0 = No, 1 = Yes)")
plt.ylabel("Bounce Rate (%)")
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "07_bounce_vs_purchase.png"),
    dpi=300
)
plt.show()

# Chart 8: Correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)
plt.title("Correlation Heatmap")
plt.tight_layout()

```

```

plt.savefig(
    os.path.join(OUTPUT_DIR, "08_correlation_heatmap.png"),
    dpi=300
)
plt.show()

# Chart 9: Page-group conversion
plt.figure(figsize=(8, 5))
page_conversion.plot(kind="bar")
plt.title("Conversion Rate by Pages Viewed")
plt.ylabel("Conversion Rate (%)")
plt.xlabel("Page Group")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "09_conversion_by_pages.png"),
    dpi=300
)
plt.show()

# Chart 10: Duration-group conversion
plt.figure(figsize=(8, 5))
duration_conversion.plot(kind="bar")
plt.title("Conversion Rate by Session Duration")
plt.ylabel("Conversion Rate (%)")
plt.xlabel("Session Duration")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(
    os.path.join(OUTPUT_DIR, "10_conversion_by_duration.png"),
    dpi=300
)
plt.show()

# -----
# 17. SUMMARY REPORT
# -----
best_device = device_conversion.idxmax()
best_device_rate = device_conversion.max()

best_source = source_conversion.idxmax()
best_source_rate = source_conversion.max()

best_visitor = visitor_conversion.idxmax()
best_visitor_rate = visitor_conversion.max()

report = f"""
E-COMMERCE CONVERSION RATE ANALYSIS REPORT
=====

Total Sessions          : {total_sessions}
Total Purchases         : {total_purchases}
Overall Conversion Rate : {conversion_rate:.2f}%

Cart Addition Rate      : {cart_rate:.2f}%
Cart-to-Purchase Rate   : {cart_to_purchase_rate:.2f}%

Highest Conversion Device:
{best_device} -> {best_device_rate:.2f}%

Highest Conversion Traffic Source:
{best_source} -> {best_source_rate:.2f}%

Highest Conversion Visitor Type:
{best_visitor} -> {best_visitor_rate:.2f}%

Average Metrics by Purchase Status:
{engagement.round(2).to_string()}

IMPORTANT:
These results are based on the generated sample dataset.
For a real project, replace the generated data with actual
e-commerce website/session data.
"""

with open(
    os.path.join(OUTPUT_DIR, "final_report.txt"),
    "w",
    encoding="utf-8"
) as f:
    f.write(report)

```

```

print("\n" + report)

# -----
# 18. EXPORT ANALYTICS TABLES
# -----
device_conversion.rename(
    "conversion_rate_percent"
).to_csv(
    os.path.join(OUTPUT_DIR, "device_conversion.csv")
)

visitor_conversion.rename(
    "conversion_rate_percent"
).to_csv(
    os.path.join(OUTPUT_DIR, "visitor_conversion.csv")
)

source_conversion.rename(
    "conversion_rate_percent"
).to_csv(
    os.path.join(OUTPUT_DIR, "source_conversion.csv")
)

correlation.to_csv(
    os.path.join(OUTPUT_DIR, "correlation_matrix.csv")
)

print("\nAll datasets, charts and reports were saved in:")
print(OUTPUT_DIR)

# =====
# END OF PROJECT
# =====

```

5. Main Formulas Used

Metric	Formula
Conversion Rate	$(\text{Number of Purchases} / \text{Total Sessions}) \times 100$
Cart Addition Rate	$(\text{Number of Sessions with Cart Added} / \text{Total Sessions}) \times 100$
Cart-to-Purchase Rate	$(\text{Purchases from Cart Users} / \text{Cart Users}) \times 100$

6. Expected Output

After running the program, an **ecommerce_outputs** folder is created. It contains the generated CSV dataset, conversion-rate tables, correlation matrix, final text report, and ten PNG charts. The terminal also displays the overall conversion rate and segment-wise conversion rates.

7. Project Interpretation

The analysis can be used to understand where website visitors are converting and where customers are dropping out. Device, visitor type, traffic source, pages viewed, session duration, bounce rate and cart activity can be compared to identify useful patterns. A higher conversion rate in a segment indicates that a larger proportion of sessions in that segment resulted in purchases; it does not by itself prove that the segment caused the higher conversion.

8. Suggested Project Presentation Points

1. Problem: E-commerce businesses receive many visitors but only a small proportion may purchase.
2. Objective: Measure conversion rate and identify customer-behavior patterns related to purchase.
3. Tools: Python, Pandas, NumPy, Matplotlib and Seaborn.
4. Process: Data generation/collection → cleaning → EDA → conversion calculation → segmentation → visualization.
5. Key metrics: Conversion Rate, Cart Rate, Cart-to-Purchase Rate, Bounce Rate and Session Duration.
6. Output: Analytical tables, charts and a final report.
7. Future enhancement: Connect the project to real website analytics data and build an interactive dashboard.

9. Important Note

This PDF contains a complete runnable project using a generated sample dataset. For an academic demonstration, it is useful because every step can be reproduced. For a real business analysis, replace the sample-data section with an actual, properly anonymized e-commerce dataset and preserve the same cleaning, analysis and visualization pipeline.